Task-07

Code:

```
import math
import numpy as np
GOS = 0.02         # 2% Grade of Service (Blocking target)
RMAX = 3.0         # 3 km maximum macrocell radius
SQ3 = math.sqrt(3) # Hexagonal geometric constant
def sample_in_hex(rng, m):
    """Generates random (x, y) coordinates uniformly distributed inside a standard hexagon."""
    points = []
    while len(points) < m:
        # Sample points in a bounding box enclosing the unit hexagon
        x = rng.uniform(-SQ3/2, SQ3/2)
        y = rng.uniform(-1.0, 1.0)
        # Verify if the coordinates fall inside the boundaries of a regular hexagon
        if abs(y) <= 1.0 and abs(x) * math.sqrt(3) + abs(y) <= 2.0:
            points.append([x, y])
    return np.array(points)
def hex_id(P, sub_r, origin_x=0, origin_y=0):
    """Maps user coordinates to spatial index keys based on sub-cell spacing configurations."""
    # Maps Cartesian points onto an integer grid coordinate system
    grid_x = np.floor((P[:, 0] - origin_x) / (sub_r * SQ3)).astype(int)
    grid_y = np.floor((P[:, 1] - origin_y) / (sub_r * 1.5)).astype(int)
    return np.column_stack((grid_x, grid_y))
def loads_by_cell(P, R, erl):
    """Simulates localized load pooling for shrinking radius layouts."""
    ids = hex_id(P, R)
    unique_ids, counts = np.unique(ids, axis=0, return_counts=True)
    loads = counts * erl
    return loads, unique_ids
def system_blocking(loads, cap_channels):
    """Calculates overall weighted average traffic blocking rate across the framework."""
    if len(loads) == 0:
        return 0.0
    blocked_traffic = sum(l * erlang_b(l, cap_channels) for l in loads)
    return blocked_traffic / sum(loads)
def erlang_b(E, c):
    B = 1.0
    for k in range(1, int(c) + 1):
        B = E * B / (k + E * B)
    return B
def erlang_cap(c, gos=GOS):
    """max offered traffic (Erlang) for c channels at blocking<=gos (bisection)"""
```

```
    if c <= 0: return 0.0
    lo, hi = 0.0, float(c) * 3 + 10
    for _ in range(60):
        m = (lo + hi) / 2
        if erlang_b(m, c) <= gos: lo = m
        else: hi = m
    return lo

def hex_cells(rings):
    out = []
    for a in range(-rings, rings + 1):
        for b in range(-rings, rings + 1):
            if abs(a + b) <= rings: out.append((a, b))
    return out
def xy(a, b, R=1.0):
    return (SQ3 * R * (a + b / 2.0), 1.5 * R * b)

def color_fn(N, i, j):
    """coset id of the sub-lattice generated by u=(i,j), v=(-j,i+j); cells with equal id are
co-channel"""
    if N == 1: return lambda a, b: 0
    return lambda a, b: (((i + j) * a + j * b) % N, ((-j) * a + i * b) % N)

def sir_mc(N, i, j, n, sigma=6.0, sector=0, m=5000, seed=1, rings=3, back_db=-20.0):
    """SIR (dB) samples for mobiles uniform in the central cell. sector in {0,120,60}."""
    rng = np.random.default_rng(seed); col = color_fn(N, i, j)
    cells = hex_cells(rings); c0 = col(0, 0)
    co = [(a, b) for (a, b) in cells if (a, b) != (0, 0) and col(a, b) == c0]
    P = sample_in_hex(rng, m)
    d0 = np.maximum(np.hypot(P[:, 0], P[:, 1]), 1e-3)
    S = d0**(-n) * 10**(rng.normal(0, sigma, m) / 10)
    I = np.zeros(m)
    nsec = 0 if sector == 0 else int(360 / sector)
    if sector:
        ang = np.degrees(np.arctan2(P[:, 1], P[:, 0])) % 360
        serv = np.floor(((ang + sector / 2) % 360) / sector)
    for (a, b) in co:
        cx, cy = xy(a, b); dx = P[:, 0] - cx; dy = P[:, 1] - cy; d = np.hypot(dx, dy)
        g = np.ones(m)
        if sector:
            bear = np.degrees(np.arctan2(dy, dx)) % 360
            diff = np.abs(((bear - serv * sector) + 180) % 360 - 180)
            g = np.where(diff <= sector / 2, 1.0, 10**(back_db / 10))
        I += g * d**(-n) * 10**(rng.normal(0, sigma, m) / 10)
```

```
    return 10 * np.log10(S / I)

def sir_equi(N, n, k=6):
    return 10 * math.log10((3 * N)**(n / 2) / k)

def sir_worst_edge(N, n):
    Q = math.sqrt(3 * N)
    return 10 * math.log10(1 / (2 * (Q - 1)**(-n) + 2 * Q**(-n) + 2 * (Q + 1)**(-n)))

def split_recursive(P, erl, R, c, level=0, maxlevel=3, gos=GOS, origin=(0, 0)):
    load = len(P) * erl
    if load <= erlang_cap(c, gos) or level >= maxlevel or len(P) == 0:
        return [(level, load)]
    ids = hex_id(P, R / 2.0, *origin)
    keys, inv = np.unique(ids, axis=0, return_inverse=True)
    inv = inv.ravel()
    out = []
    for k in range(len(keys)):
        out += split_recursive(P[inv == k], erl, R / 2.0, c, level + 1, maxlevel, gos, origin)
    return out

def min_sites(P, erl, cap_channels, gos=GOS, cells_cap=None, step=0.985):
    R = RMAX
    while R > 0.01:
        L, _ = loads_by_cell(P, R, erl)
        b = system_blocking(L, cap_channels)
        if b <= gos:
            return R, len(L), b, max(erlang_b(l, cap_channels) for l in L)
        R *= step
    return R, None, None, None
if __name__ == "__main__":
    print(f"--- Erlang Engine Verification ---")
    capacity_2percent = erlang_cap(c=30, gos=0.02)
    print(f"30 Channels at 2% GoS can sustain an operational load of: {capacity_2percent:.2f}
Erlangs\n")

    print(f"--- Running Monte Carlo SIR Analysis ---")
    # Using N=7 cluster rules (i=2, j=1, Path Loss Exponent=4)
    sir_samples = sir_mc(N=7, i=2, j=1, n=4)
    outage = np.mean(sir_samples < 18.0) * 100
    print(f"Mean Received SIR: {np.mean(sir_samples):.2f} dB | Outage Rate (<18dB target):
{outage:.2f}%\n")

    print(f"--- Running Dynamic Quadtree-Split Optimization ---")
```

```
rng_test = np.random.default_rng(42)
synthetic_user_coords = rng_test.uniform(-2.0, 2.0, (500, 2)) # Deploys 500 mobile nodes

layout_leaves = split_recursive(synthetic_user_coords, erl=0.03, R=1.5, c=15)
print(f"Recursive network design complete. Total sub-cells deployed: {len(layout_leaves)}")
```

Output:

```
========== RESTART: C:/AI LAB/MC assignment task.py ==========
--- Erlang Engine Verification ---
30 Channels at 2% GoS can sustain an operational load of: 21.93 Erlangs

--- Running Monte Carlo SIR Analysis ---
Mean Received SIR: 23.77 dB | Outage Rate (<18dB target): 31.72%

--- Running Dynamic Quadtree-Split Optimization ---
Recursive network design complete. Total sub-cells deployed: 16
>>
```